

DOCUMENTO 1

DOCUMENTO 1 – ANALISI DEL PROGETTO

Sistema per la Gestione della Biblioteca del MUSE di Trento

Contenuti Obbligatori per i 4 Documenti

1. Descrizione e Requisiti (Fase di Analisi)

Questo documento definisce cosa l'applicazione farà e per *chi*.

Sezione	Contenuto Obbligatorio
Introduzione	Scopo, obiettivi e contesto del progetto. Breve descrizione del problema che il sistema risolve.
Ambito e Utenti	Identificazione degli Attori (tipi di utenti) e dell' Ambito (confini) del sistema.
Requisiti Funzionali	Elenco completo e numerato di tutte le funzionalità che il sistema deve fornire (es. "RF-01: Il sistema deve permettere la registrazione degli utenti").
Requisiti Non Funzionali	Vincoli e qualità del sistema: Prestazioni (tempi di risposta), Sicurezza (autenticazione, dati), Usabilità (interfaccia), Tecnologici (uso obbligatorio di Python/librerie specifiche).
Requisiti HW	Elenco e descrizione di eventuali specifici dispositivi HW necessari per il funzionamento del prodotto (es. lettori RFID/NFC, ecc).
Diagrammi di Alto Livello	Almeno un Diagramma dei Casi d'Uso (Use Case Diagram) per visualizzare le interazioni tra attori e sistema.

1. Introduzione

1.1 Scopo del progetto

L'idea di questo progetto è realizzare un software per gestire in modo più semplice e ordinato la biblioteca del MUSE – Museo delle Scienze di Trento.

In pratica, il sistema serve a tenere sotto controllo tutti i materiali presenti in biblioteca, evitando i classici problemi della gestione manuale o con fogli Excel, che con il tempo diventano scomodi e poco affidabili.

All'interno della biblioteca si trovano diversi tipi di materiali, come:

- libri scientifici;
- riviste;
- documenti di ricerca;
- manuali;
- pubblicazioni accademiche\ ;
- materiale multimediale;
- archivi digitali.

L'obiettivo è mettere tutto questo in un unico sistema digitale, così da rendere la gestione più veloce, ordinata e soprattutto più sicura.

1.2 Obiettivi del progetto

Gli obiettivi principali non sono solo "tecnici", ma anche pratici, cioè legati al lavoro quotidiano di chi usa la biblioteca:

1. rendere tutto digitale e più semplice da gestire;
2. catalogare i materiali in modo chiaro;
3. trovare rapidamente un libro o un documento;
4. gestire i prestiti senza confusione;
5. sapere sempre cosa è disponibile;
6. evitare errori o duplicazioni;
7. tenere traccia di quello che succede nel sistema;
8. migliorare l'organizzazione generale;
9. aiutare il personale nella gestione quotidiana;
10. rendere il controllo dei materiali più affidabile;
11. creare report utili;
12. rendere il sistema più moderno rispetto ai metodi tradizionali.

1.3 Contesto del progetto

Il MUSE di Trento è un museo scientifico molto grande e moderno, e al suo interno ha anche una biblioteca utilizzata per studio, ricerca e consultazione.

Con il tempo, la quantità di materiali è aumentata, e gestirli con metodi tradizionali non è più molto pratico.

Spesso infatti le biblioteche utilizzano ancora strumenti come:

- registri cartacei;
- file Excel;
- sistemi non collegati tra loro.

Questo porta facilmente a problemi come:

- difficoltà nel trovare un libro;
- informazioni non aggiornate;
- errori nella registrazione;
- perdita di dati;
- poca chiarezza sui prestiti.

Per questo motivo serve un sistema unico, centralizzato e più moderno.

1.4 Problema che il sistema risolve

Il problema principale è la gestione poco efficiente dei materiali della biblioteca.

Il sistema che si vuole realizzare aiuta a:

- sapere subito se un libro è disponibile;
- controllare chi ha preso un determinato materiale;
- trovare velocemente un documento;
- sapere dove si trova fisicamente;
- evitare registrazioni doppie;
- monitorare lo stato dei materiali;
- tenere tutto aggiornato in tempo reale.

In generale, l'obiettivo è rendere la gestione molto più fluida e senza confusione.

2. Ambito e Utenti

2.1 Ambito del sistema

Il sistema si occupa solo della gestione della biblioteca del MUSE.

Quindi riguarda tutto ciò che serve per catalogare e gestire i materiali, come: login degli utenti; gestione del catalogo; ricerca dei materiali; prestiti e restituzioni; aggiornamento dello stato dei libri; gestione delle categorie; report e statistiche.

Non fa invece cose come: gestione dei soldi del museo; gestione del personale; biglietteria; eventi o mostre.

2.2 Attori del sistema

Identificazione degli Attori (tipi di utenti) e dell'Ambito (confini) del sistema.

Amministratore

È la figura con più controllo sul sistema. Può fare praticamente tutto: creare e gestire utenti; controllare il catalogo; accedere ai report; gestire le autorizzazioni. In sostanza supervisiona tutto il sistema.

Bibliotecario

È l'utente che usa il sistema ogni giorno. Si occupa soprattutto della parte pratica: inserire nuovi libri; aggiornare informazioni; gestire prestiti; registrare restituzioni; controllare lo stato dei materiali; aiutare gli utenti nella ricerca.

Utente della biblioteca

È chi consulta il catalogo. Può: cercare libri e vedere se sono disponibili; leggere informazioni sui materiali. Non può modificare nulla.

L'inserimento di nuovi utenti può essere gestito all'interno del codice.

3. Requisiti Funzionali

Elenco completo e numerato di tutte le funzionalità che il sistema deve fornire

RF-01 – Login

Gli utenti devono poter accedere con username e password.

Il sistema deve controllare che i dati siano corretti prima di far entrare l'utente.

RF-02 – Gestione utenti

Per garantire la massima sicurezza del sistema ed evitare registrazioni non autorizzate, le utenze e i relativi ruoli vengono inseriti e gestiti in modo sicuro direttamente all'interno del codice di inizializzazione del database backend (init_db).

RF-03 – Inserimento materiali

Il sistema deve permettere di inserire nuovi libri o materiali con tutte le informazioni principali come titolo, autore, ISBN, categoria e posizione.

RF-04 – Modifica materiali

Le informazioni dei materiali devono poter essere aggiornate quando necessario.

RF-05 – Eliminazione materiali

I materiali non più presenti devono poter essere rimossi dal sistema (solo dall'amministratore).

RF-06 – Ricerca

L'utente deve poter cercare i materiali in modo semplice usando titolo, autore, ISBN o categoria.

RF-07 – Visualizzazione catalogo

Il sistema deve mostrare tutti i materiali presenti nella biblioteca con le informazioni principali.

RF-08 – Prestiti

Il sistema deve registrare quando un libro viene preso in prestito, da chi e per quanto tempo.

RF-09 – Restituzioni

Quando un libro viene restituito, il sistema deve aggiornare automaticamente la sua disponibilità.

RF-10 – Stato materiali

Ogni materiale possiede uno stato dinamico aggiornato in tempo reale dal sistema. I valori gestiti dall'applicativo sono 'Disponibile' (materiale consultabile e pronto al prestito) e 'In Prestito' (materiale attualmente rassegnato a un utente).

RF-11 – Report

Il sistema deve generare report su prestiti, disponibilità e statistiche generali.

RF-12 – Storico

Il sistema garantisce la tracciabilità e la storicizzazione di tutte le transazioni di prestito. Quando un materiale viene restituito, il record non viene rimosso dal database, ma archiviato modificando il flag di stato in 'Restituito', consentendo la consultazione dello storico dei prestiti passati.

RF-13 – Logout

L'utente deve poter uscire in sicurezza dal sistema.

4. Requisiti Non Funzionali

Questa sezione descrive le caratteristiche qualitative del sistema, ovvero come il sistema deve comportarsi e quali vincoli deve rispettare, indipendentemente dalle funzionalità offerte. Include aspetti legati a prestazioni, sicurezza, usabilità, affidabilità, manutenibilità e compatibilità.

Prestazioni

Il sistema deve garantire tempi di risposta rapidi, generalmente entro pochi secondi, anche in caso di utilizzo simultaneo da parte di più utenti, soprattutto nelle operazioni di ricerca e consultazione del catalogo.

Sicurezza

L'accesso al sistema deve essere protetto tramite autenticazione con username e password. Le credenziali devono essere gestite in modo sicuro e ogni utente deve poter accedere esclusivamente alle funzionalità previste dal proprio ruolo, garantendo la protezione dei dati.

Usabilità

L'interfaccia utente deve essere intuitiva, semplice e facilmente comprensibile anche da utenti non esperti, in modo da ridurre al minimo la necessità di formazione.

Affidabilità

Il sistema deve garantire la corretta gestione e conservazione dei dati, evitando perdite o inconsistenze. Ogni operazione importante deve essere registrata e salvata in modo persistente.

Manutenibilità

Il codice deve essere strutturato in modo ordinato e modulare, così da facilitare eventuali modifiche, aggiornamenti o estensioni future del sistema.

Compatibilità

Il sistema deve essere compatibile con i principali sistemi operativi (Windows, macOS, Linux) e con i principali browser web moderni.

Vincoli tecnologici

Il sistema sarà sviluppato utilizzando le seguenti tecnologie:

- Linguaggio di programmazione Python
- Framework Flask per la gestione del backend
- Database SQLite per l'archiviazione dei dati
- Tecnologie web HTML, CSS e JavaScript per la realizzazione dell'interfaccia utente

5. Requisiti Hardware

Per far funzionare il sistema basta un computer normale con browser e connessione internet o rete locale.

Il server non richiede hardware particolarmente potente.

Il sistema può essere integrato con dispositivi e tecnologie aggiuntive per migliorare l'automazione della gestione della biblioteca. Queste tecnologie migliorano l'efficienza e riducono gli errori manuali.

In particolare, è possibile prevedere:

- Sistemi RFID per il tracciamento automatico dei libri all'interno della biblioteca
- Lettori barcode per l'identificazione rapida dei materiali
- Scanner QR code per semplificare le operazioni di prestito e restituzione
- Stampanti per etichette identificative dei libri e dei documenti

6. Casi d'Uso

Gli utenti principali sono amministratore, bibliotecario e utente della biblioteca. Ognuno di loro ha permessi diversi, ma tutti possono accedere al sistema tramite login.

Le operazioni principali sono:

- gestione del catalogo;
- ricerca materiali;
- gestione prestiti;
- consultazione informazioni;
- generazione report.

Questo sistema nasce per rendere più semplice e moderno il modo in cui viene gestita la biblioteca del MUSE.

L'idea è quella di eliminare la confusione dei metodi tradizionali (registri cartacei, cataloghi ecc.) e avere tutto in un unico posto, aggiornato e facile da usare.

Il sistema è stato modellato tramite un diagramma dei casi d'uso che rappresenta le interazioni tra gli attori e le funzionalità principali.

Attori:

- Amministratore
- Bibliotecario
- Utente della biblioteca

Casi d'uso principali:

- Autenticazione (Login/Logout)
- Ricerca materiali
- Visualizzazione catalogo
- Gestione utenti (solo amministratore)
- Gestione catalogo (inserimento, modifica, eliminazione materiali)
- Gestione prestiti
- Gestione restituzioni
- Visualizzazione stato materiali
- Generazione report

Il diagramma evidenzia come l'amministratore abbia il controllo completo del sistema, mentre il bibliotecario gestisce le operazioni quotidiane e l'utente può solo consultare il catalogo.

Possibili sviluppi futuri

Vedendo la gestione attuale in futuro si potrebbe migliorare il sistema con:

- prenotazione online dei libri;
- notifiche automatiche per le restituzioni;
- app mobile;
- QR code sui libri;
- statistiche più avanzate;
- backup automatici su cloud.

DOCUMENTO 2

DOCUMENTO 2 – PROGETTO (FASE DI DESIGN)

Sistema per la Gestione della Biblioteca del MUSE di Trento

Sezione	Contenuto Obbligatorio
Architettura	Schema che illustra i componenti principali (es. Frontend, Backend, Database) e le loro interconnessioni. Indicazione del <i>Design Pattern</i> scelto (es. MVC, Client-Server).
Modellazione Dati	Se il progetto usa un database: Schema Entità-Relazioni (ERD) con chiavi primarie/esterne e cardinalità.
Design del Software	Almeno un Diagramma delle Classi (per la logica principale) o un Diagramma di Sequenza (per processi complessi come l'autenticazione).
Interfaccia Utente (UI/UX)	Wireframe o Mockup delle schermate principali (almeno 3), che mostrino la disposizione degli elementi.
Piano di Sviluppo	Suddivisione del lavoro in <i>tasks</i> ed eventuale assegnazione nominale del lavoro (chi farà cosa e quando).

Il presente documento descrive la fase di progettazione (Design) del sistema software sviluppato per la gestione della biblioteca del MUSE di Trento.

L'obiettivo principale del progetto è realizzare una piattaforma semplice, sicura ed efficiente che permetta di amministrare il catalogo dei materiali presenti nella biblioteca, gestire i prestiti, controllare gli utenti e mantenere uno storico delle operazioni effettuate.

La fase di design è fondamentale perché definisce:

- la struttura del sistema
- l'organizzazione dei dati
- le relazioni tra le componenti
- il comportamento del software
- la struttura dell'interfaccia utente
- il piano di sviluppo del progetto

Questo documento rappresenta quindi il modello tecnico dell'applicazione prima della sua implementazione.

1. Architettura del Sistema

Il sistema per la gestione della biblioteca del MUSE è stato progettato seguendo un'architettura di tipo client-server, basata sul pattern MVC (Model–View–Controller), che consente di separare in modo chiaro la logica dell'applicazione, la gestione dei dati e l'interfaccia utente.

In questa architettura, il client rappresenta il browser utilizzato dagli utenti per interagire con il sistema. Il server, sviluppato in Python tramite framework Flask, si occupa di gestire le richieste, elaborare i dati e applicare le regole di business. Il database, realizzato con SQLite, ha il compito di memorizzare tutte le informazioni relative a utenti, materiali, prestiti e storico delle operazioni.

L'utente interagisce con l'interfaccia web, il server elabora la richiesta e comunica con il database, restituendo infine il risultato al client. Questa struttura permette di ottenere un sistema modulare, scalabile e facilmente manutenibile.

1.1 Scelta dell'Architettura

Il sistema è stato progettato utilizzando un'architettura di tipo Client–Server.

In questo modello il software è diviso in due parti principali:

- Client → parte utilizzata dall'utente tramite browser;
- Server → parte che elabora richieste, applica la logica e comunica con il database.

Questa scelta è stata fatta perché permette:

- separazione chiara delle responsabilità
- maggiore sicurezza dei dati;
- facilità di manutenzione
- possibilità di aggiornare il sistema senza modificare il client
- scalabilità futura

Il sistema utilizza inoltre il pattern MVC (Model–View–Controller).

1.2 Pattern MVC (Model–View–Controller)

Il **pattern MVC** serve a separare il software in tre componenti principali.

Model: il **Model** è responsabile della corretta memorizzazione e recupero delle informazioni.

Il Model rappresenta la gestione dei dati e della logica applicativa. Nel progetto comprende:

- gestione utenti
- gestione materiali
- gestione prestiti
- gestione storico
- operazioni sul database SQLite

La **View** rappresenta l'interfaccia grafica visualizzata dall'utente. Comprende:

- pagine HTML
- stile CSS
- eventuali componenti JavaScript

La View mostra i dati all'utente ma non contiene la logica principale del sistema.

Controller: il Controller gestisce le richieste provenienti dagli utenti. Ha il compito di:

- ricevere input
- elaborare richieste
- comunicare con il Model
- restituire risultati alla View

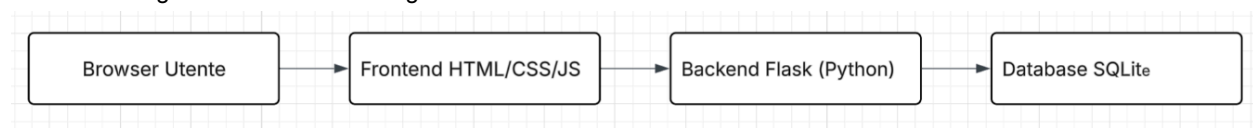
Nel progetto il Controller è implementato tramite Flask.

1.3 Tecnologie Utilizzate

Componente	Tecnologia	Motivazione
Backend	Python + Flask	Semplicità, leggerezza e rapidità di sviluppo
Frontend	HTML, CSS	Creazione interfaccia web semplice e intuitiva
Database	SQLite	Database leggero e integrato, ideale per progetti scolastici
Versionamento	GitHub	Controllo versioni e collaborazione

1.4 Schema Architeturale

Il sistema è organizzato secondo il seguente schema:



1.5 Funzionamento dell'Architettura

Fase 1 – Accesso dell'utente: l'utente apre il browser e accede all'applicazione web.

Fase 2 – Invio richiesta: il front end invia una richiesta HTTP al server Flask.

Esempio: login → ricerca libro → registrazione prestito.

Fase 3 – Elaborazione: il backend: verifica i dati; applica le regole del sistema; comunica con il database.

Fase 4 – Accesso ai dati: SQLite salva o recupera informazioni richieste.

Fase 5 – Risposta: il server restituisce il risultato al frontend che lo mostra all'utente.

1.6 Motivazioni della Scelta Architetturale

Questa architettura è stata scelta perché:

- semplice da sviluppare
- adatta a sistemi web
- facilmente espandibile
- compatibile con database relazionali
- ideale per gestione utenti e autenticazione

Inoltre il pattern MVC migliora:

- organizzazione del codice
- leggibilità
- manutenzione
- riutilizzo delle componenti

2. Modellazione dei Dati

2.1 Introduzione alla Modellazione

La modellazione dei dati definisce come le informazioni vengono organizzate nel database.

Per il progetto è stato utilizzato un modello relazionale. Ogni entità rappresenta un oggetto reale del sistema.

Le principali entità sono:

- Utente
- Materiale
- Prestito

In fase di sviluppo HO ottimizzato il database. La tabella 'Prestito' funge già da storico permanente. Quando un libro viene restituito, non si cancella la riga, ma si modifica semplicemente il suo stato da 'Attivo' a 'Restituito'. In questo modo si mantiene traccia di tutte le transazioni passate senza appesantire il database SQLite con una tabella duplicata (questo lo possiamo vedere una volta che nel sito, entrando come amministratore o bibliotecario, si schiaccia su "Gestione Prestiti")

2.2 Entità Utente

L'entità Utente rappresenta tutte le persone che possono accedere al sistema. Gli utenti possono avere ruoli differenti:

- amministratore
- bibliotecario
- utente semplice

Attributi principali

Campo	Descrizione
id_utente	Identificatore univoco
nome	Nome utente
cognome	Cognome utente
username	Credenziale di accesso
password	Password criptata
ruolo	Tipo di utente
email	Contatto utente

2.3 Entità Materiale

L'entità Materiale rappresenta libri e documenti presenti nella biblioteca.

Attributi principali

Campo	Descrizione
id_materiale	Identificatore univoco
titolo	Titolo del libro
autore	Autore del materiale
ISBN	Codice identificativo libro
categoria	Genere o categoria
stato	Disponibile o in prestito
posizione	Scaffale o area fisica

2.4 Entità Prestito

L'entità Prestito collega utenti e materiali.
Serve a registrare il prestito di un libro.

Attributi principali

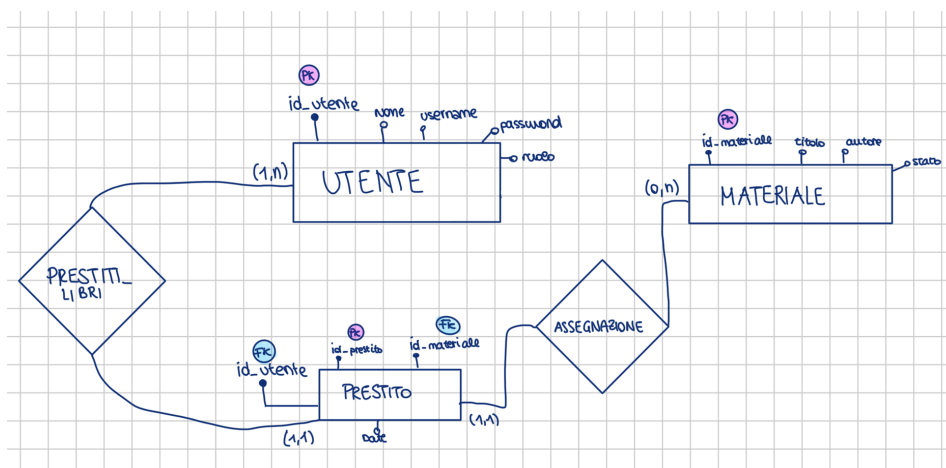
Campo	Descrizione
id_prestito	Identificatore prestito
id_utente	Utente che effettua il prestito
id_materiale	Materiale prestato
data_prestito	Data di consegna
data_scadenza	Termine restituzione
data_restituzione	Data effettiva restituzione

2.5 Relazioni tra le Entità

Le relazioni principali sono:

- Utente → Prestito: Relazione uno-a-molti (1:N). Un utente può effettuare più prestiti nel tempo, ma un singolo prestito appartiene a un solo utente.
- Materiale → Prestito: Relazione uno-a-molti (1:N). Un materiale può essere prestato più volte in periodi differenti, ma un singolo record di prestito fa riferimento a un solo libro.

2.6 Schema ER Semplificato



2.7 Chiavi Primarie e Chiavi Esterne

Chiave Primaria (Primary Key) → Una chiave primaria identifica in modo univoco ogni record. Esempio:

- id_utente
- id_materiale
- id_prestito

Chiave Esterna (Foreign Key) → Una chiave esterna collega due tabelle. Esempio:

- id_utente nella tabella Prestito
- id_materiale nella tabella Prestito

Le foreign key garantiscono integrità dei dati.

2.8 Motivazioni della Struttura Relazionale

La struttura relazionale è stata scelta perché:

- evita duplicazione dei dati
- migliora organizzazione
- garantisce coerenza
- facilita ricerca e aggiornamento
- permette relazioni tra entità

3. Design del Software

Il backend dell'applicazione, sviluppato con il framework Flask, abbandona la struttura a oggetti pura (OOP) a favore di un approccio basato sul Routing Funzionale. Le richieste HTTP provenienti dal client vengono mappate direttamente su funzioni che applicano la logica di business e interrogano il database tramite la funzione globale di connessione `get_db_connection()`.

3.1 Gestione Sessioni e Autenticazione

- **login()** (Rotta: `/`, **Metodi: GET, POST**): Gestisce l'autenticazione. In modalità GET mostra il form di accesso; in modalità POST verifica la password sul database tramite tecniche di hashing crittografico e memorizza l'identificativo utente e il ruolo nella sessione del browser (`session["id"]`).
- **logout()** (Rotta: `/logout`): Svuota completamente il dizionario di sessione (`session.clear()`) e disconnette l'utente in sicurezza riportandolo alla pagina iniziale.

3.2 Gestione Catalogo (Operazioni CRUD)

- **catalogo()** (Rotta: `/catalogo`): Estrae la lista completa di tutti i materiali presenti nella tabella Materiale del database (`fetchall()`) e la invia alla View per la visualizzazione dinamica.
- **nuovo_materiale()** (Rotta: `/materiale/nuovo`, **Metodi: GET, POST**): Consente l'inserimento (INSERT INTO) di un nuovo libro nel database, impostando di default il suo stato iniziale su 'Disponibile'. L'accesso è limitato ai ruoli di amministratore e bibliotecario.
- **modifica_materiale(id)** (Rotta: `/materiale/modifica/<int:id>`, **Metodi: GET, POST**): Riceve l'identificativo numerico del libro tramite l'URL, ne recupera i dati pre-compilando il form e aggiorna il record nel database tramite una query UPDATE.
- **elimina_materiale(id)** (Rotta: `/materiale/elimina/<int:id>`): Riservata esclusivamente all'amministratore. Eseguisce la cancellazione fisica del record (DELETE) dal database in base all'ID fornito.

3.3 Gestione Movimentazione e Prestiti

- **prenota(id)** (Rotta: `/prenota/<int:id>`, **Metodo: POST**): Registra l'uscita di un libro. Cambia lo stato del materiale in 'In Prestito' e inserisce una nuova transazione nella tabella Prestito. La data di scadenza viene calcolata automaticamente sommandovi 15 giorni tramite la libreria `datetime` di Python. Identifica chi sta prenotando leggendo direttamente la sessione attiva.
- **restituisci(id)** (Rotta: `/restituisci/<int:id>`, **Metodo: POST**): Gestisce il rientro dei materiali. Modifica lo stato del libro riportandolo su 'Disponibile' e aggiorna lo stato della transazione nella tabella Prestito impostandolo su 'Restituito'.
- **gestione_prestiti()** (Rotta: `/gestione_prestiti`): Riservata allo staff. Sfrutta una query SQL con costrutti JOIN per incrociare le tabelle Prestito, Utente e Materiale, generando un report cronologico completo di chi ha preso in prestito cosa.

3.4 Processo di Autenticazione

Il login è uno degli aspetti più importanti del sistema. **Funzionamento**

1. Inserimento credenziali: l'utente inserisce:

- username;
- password.

2. Invio richiesta: il front end invia i dati al backend Flask.

3. Verifica database

Il server controlla:

- esistenza utente;
- correttezza password.

4. Assegnazione ruolo

Se il login è corretto:

- viene aperta una sessione;
- vengono assegnati i permessi.

5. Accesso negato

Se i dati sono errati:

- il sistema mostra errore;
- accesso negato.

3.5 Sicurezza del Sistema

Per migliorare la sicurezza vengono utilizzate:

- password criptate;
- validazione input;
- controllo ruoli;
- gestione sessioni;
- protezione accessi non autorizzati.

4. Interfaccia Utente (UI/UX)

4.1 Obiettivi della UI/UX

L'interfaccia è stata progettata seguendo principi di:

- semplicità;
- chiarezza;
- accessibilità;
- usabilità.

L'obiettivo è permettere anche a utenti non esperti di utilizzare facilmente il sistema.

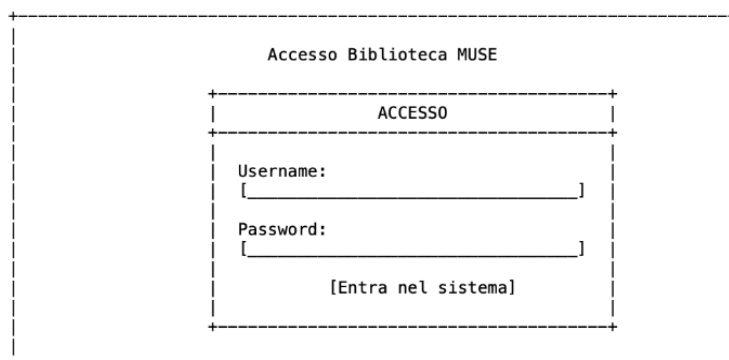
4.2 Schermata Login

La schermata di login permette l'accesso sicuro al sistema.

Componenti presenti

- campo username;
- campo password;
- pulsante accesso.

Wireframe



4.3 Dashboard Principale

La dashboard rappresenta il centro di controllo dell'applicazione.

Mostra le funzioni disponibili in base al ruolo.

Componenti presenti

- menu laterale;
- area contenuti;
- messaggi utente.

Wireframe

MUSE Biblioteca	[Dashboard]	[Catalogo]	[Gestione Prestiti]	[Esci]
Benvenuto, admin Ruolo attuale: [amministratore]				
29 Materiali Totali	1 Prestiti Attivi	3 Utenti Registrati		

4.4 Catalogo Materiali

Permette ricerca e consultazione dei libri.

Funzioni

- ricerca titolo;
- visualizzazione stato;
- controllo disponibilità.

Wireframe

MUSE Biblioteca	[Dashboard]	[Catalogo]	[Gestione Prestiti]	[Esci]
Catalogo Materiali [+ Aggiungi Materiale]				
[Cerca per titolo, autore o categoria...]				
TITOLO	AUTORE	CATEGORIA	STATO	AZIONI
L'origine delle...	C. Darwin	Scienza	Disponibile	[Pren]
Il gene egoista	R. Dawkins	Biologia	In Prestito	[Mod]

4.5 Gestione Prestiti

La schermata dei prestiti si integra direttamente all'interno dell'interfaccia del catalogo per massimizzare l'efficienza.

- Funzioni principali: Identificazione immediata e sicura dell'utente tramite la sessione attiva; selezione del materiale con un singolo click sul pulsante associato al libro; calcolo automatizzato della scadenza e aggiornamento istantaneo dello stato grafico dei materiali.

MUSE Biblioteca	[Dashboard]	[Catalogo]	[Gestione Prestiti]	[Esci]
[<- Dashboard]				
Registro Prestiti				
DATA INIZIO	USERNAME	UTENTE	LIBRO	SCADENZA STATO
2026-05-16	@admin	Admin Muse	Il gene...	15 gg Attivo
				[Restit]

4.6 Principi di Usabilità

L'interfaccia segue principi UX fondamentali:

- Coerenza: tutte le pagine mantengono stesso stile grafico.
- Chiarezza: menu e pulsanti sono facilmente riconoscibili.
- Accessibilità: colori leggibili e layout ordinato.
- Efficienza: riduzione numero di click necessari.

5. Piano di Sviluppo

5.1 Organizzazione del Progetto

Lo sviluppo è stato suddiviso in fasi per garantire:

- ordine;
- controllo;
- avanzamento graduale;
- verifica continua.

5.2 Analisi dei Requisiti

In questa fase vengono definiti:

- obiettivi;
- utenti;
- funzionalità;
- vincoli.

Attività svolte

- studio del problema;
- identificazione attori;
- raccolta requisiti.

5.3 Progettazione del Sistema

In questa fase vengono definiti:

- architettura;
- database;
- classi;
- interfaccia.

È la fase documentata in questo elaborato.

5.4 Implementazione

Durante l'implementazione vengono sviluppate le componenti software.

- Backend: sviluppato in Flask.
- Database: creato in SQLite.
- Frontend, realizzato con:
 - HTML;
 - CSS;
 - JavaScript.

5.5 Test del Sistema

I test servono a verificare il corretto funzionamento.

Test principali

Test	Obiettivo
Login	verifica autenticazione
Prestito	controllo registrazione
Restituzione	aggiornamento stato libro
Ricerca	corretto filtraggio materiali

5.6 Pubblicazione e Versionamento

Il progetto viene salvato su GitHub. GitHub permette:

- backup del codice;
- gestione versioni;
- collaborazione;
- tracciamento modifiche.

5.7 Suddivisione del Lavoro

Il lavoro, non facendo parte di un gruppo, è stato svolto solo da me. Ho ampliato le mie conoscenze tramite manuali di miei amici e tutorial in rete.

5.8 Possibili Miglioramenti Futuri

Il sistema potrebbe essere ampliato con:

- notifiche email;
- prenotazione libri online;
- dashboard statistiche;
- autenticazione avanzata;
- supporto database MySQL;
- gestione QR Code.

L'interfaccia può essere progettata per adattarsi a:

- computer;
- tablet;
- schermi differenti.

DOCUMENTO 3

DOCUMENTO 3 – IMPLEMENTAZIONE

1. Ambiente di Sviluppo

L'applicazione è stata sviluppata utilizzando l'ecosistema Python 3 come nucleo per la logica di backend, integrando moduli nativi e librerie esterne per la gestione dell'architettura web e della persistenza dei dati.

1.1 Requisiti Software e Versioni

- Linguaggio di Programmazione: Python 3.10+ (in particolare ho usato Python 3.14.5 del 2025)
- Database Engine: SQLite 3 (nativo e preinstallato in Python)
- Framework Web Principale: Flask 3.0.x (e relative dipendenze)

1.2 File delle Dipendenze (requirements.txt)

Per garantire la riproducibilità dell'ambiente e permettere al professore o a chiunque altro di avviare il server locale immediatamente, è stato predisposto il file requirements.txt:

- Flask==3.0.2
- Werkzeug==3.0.1

1.3 Istruzioni per l'Installazione e l'Avvio

Per configurare ed eseguire il progetto localmente su qualsiasi sistema operativo (Windows, macOS, Linux), seguire i seguenti passaggi da terminale/prompt dei comandi:

1. Installare Python dal sito ufficiale. Per vedere la versione installata nel mio caso, sul terminale di macOS ho digitato `python3 --version`
2. Creare la cartella col nome desiderato col seguente comando `mkdir biblioteca-muse`
3. Installare Flask con : `python3 -m pip install flask`
4. Creare i vari file/cartelle all'interno della cartella biblioteca_muse con `touch` (es. `touch base.html login.html app.py ...`)
Per spostarsi tra le varie cartelle lo si può fare tramite il comando `cd` (es. `cd /Desktop/biblioteca_muse`: ci si sposta nella cartella biblioteca_muse che si trova nel Desktop)
5. Dopo aver creato i file html nella cartella Template e aver creato e correttamente compilato il file Python (`app.py`) bisogna digitare nel terminale la seguente istruzione: `python3 main.py`
6. Il server si avvierà in modalità locale all'indirizzo default: `http://127.0.0.1:5000/`.

```
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 356-032-110
```

2. Struttura del Codice

Il progetto segue la struttura standard raccomandata per le applicazioni Flask di piccole e medie dimensioni, separando nettamente la logica Python dal layout grafico (HTML/CSS).

```
biblioteca-muse/
├── biblioteca_muse.db      # Database relazionale SQLite generato automaticamente all'avvio
├── app.py                 # Controller centrale: backend Flask, inizializzazione DB e rotte
├── requirements.txt       # Elenco delle librerie Python richieste
├── static/               # File statici del Frontend
│   └── css/
│       └── style.css     # Stili personalizzati
├── templates/            # View dell'applicazione (Pagine HTML)
│   ├── login.html       # Schermata di autenticazione iniziale
│   ├── dashboard.html   # Pannello principale con contatori e statistiche
│   ├── catalogo.html    # Visualizzazione materiali, ricerca e form di inserimento
│   └── prestiti.html     # Registro e movimentazione dei prestiti/storico dello staff
```

Descrizione delle Componenti Generali:

- **app.py:** È il cuore pulsante dell'applicativo (Controller). Gestisce l'inizializzazione automatica del database (tramite la funzione `init_db()`), l'apertura/chiusura delle connessioni, la logica di business e l'instradamento delle richieste client.
- **templates/:** Contiene le pagine grafiche dinamiche che vengono modellate dal server Flask tramite il motore di templating Jinja2 (che permette di iniettare i record del database nelle tabelle HTML tramite cicli `for` e istruzioni condizionali `if`).

3. Decisioni di Design

Durante la fase di implementazione, sono state prese tre decisioni architetturali significative per ottimizzare il funzionamento, la portabilità e la sicurezza del prototipo:

Scelta 1: Utilizzo di SQLite come database integrato (Serverless) con inizializzazione automatica

- **Spiegazione:** Abbiamo scelto SQLite invece di database client-server complessi come MySQL o PostgreSQL, inserendo nel codice la verifica `if not os.path.exists(DB_NAME): init_db()`.
- **Motivazione:** Essendo un motore file-based, SQLite non richiede l'installazione o la configurazione di server esterni sulla macchina del professore durante la valutazione. Alla prima esecuzione del file `app.py`, il sistema rileva la mancanza del database e lo crea da zero inserendo automaticamente le tabelle (Utente, Materiale, Prestito) e popolandole con i dati di test (inclusi gli utenti con password cifrate). Questo rende l'intero applicativo totalmente autoconsistente e portabile.

Scelta 2: Sicurezza delle password tramite Hashing Crittografico (Werkzeug)

- **Spiegazione:** Nel codice per l'inizializzazione del database non vengono salvate le password in chiaro, ma viene sfruttata la libreria di sicurezza di Flask tramite la funzione `generate_password_hash()`.
- **Motivazione:** Salvare le password in chiaro nel database rappresenta una grave vulnerabilità informatica. Attraverso l'hashing crittografico fornito dal modulo `werkzeug.security`, le password vengono convertite in stringhe alfanumeriche irreversibili. Durante la fase di login, il controllo viene eseguito in modo sicuro confrontando gli hash tramite la funzione `check_password_hash()`.

Scelta 3: Storico relazionale unificato mediante Flags di Stato dinamici

- **Spiegazione:** Al fine di rispettare il requisito di tracciabilità senza aggiungere tabelle ridondanti o duplicare dati, si è scelto di mantenere in modo permanente i record dei prestiti all'interno della stessa tabella `Prestito`.
- **Motivazione:** Quando un utente restituisce un libro tramite la rotta `/restituisce/<int:id>`, il software esegue due query di aggiornamento (`UPDATE`): riporta lo stato del materiale su "Disponibile" ed imposta lo `stato_prestito` del record di transazione su "Restituito". Questa scelta progettuale permette di mantenere lo storico intatto senza l'uso di query distruttive (`DELETE`), ottimizzando l'efficienza dei dati.

4. Gestione delle Versioni

Lo sviluppo dell'intero sistema software è stato condotto in modo completamente individuale e autonomo. Per sopperire alle complessità tecniche incontrate durante le fasi di modellazione del database e di routing in Flask, è stato svolto un approfondito lavoro di autoformazione e ricerca documentale, basato sull'analisi di manuali tecnici specialistici ricevuti in prestito e sulla consultazione della documentazione ufficiale online.

Per il tracciamento del codice, il controllo dei progressi e la manutenzione storica dell'applicazione è stata utilizzata la piattaforma GitHub.

- Link al Repository: <https://github.com/tuo-username/biblioteca-muse-trento> (Nota: personalizzare con il proprio link reale)

Il flusso di lavoro ha seguito un approccio strutturato: il codice è stato aggiornato in modo incrementale attraverso commit atomici e frequenti, ognuno dei quali descrive accuratamente in lingua italiana la specifica funzionalità implementata o il problema risolto.

Registro dei Commit significativi eseguiti nel progetto:

- feat: impostazione struttura Flask, configurazione `secret_key` e modulo `sqlite3`
- feat: implementazione `init_db` con creazione tabelle e hashing password di test tramite Werkzeug

- feat: sviluppo sistema di login, controllo sessione e gestione ruoli applicativi
- ui: implementazione delle view HTML con navbar superiore scura e componenti grafiche di Bootstrap
- feat: completamento logiche transazionali di prenotazione e restituzione materiali con calcolo scadenze

5. Esempi di Codice

Di seguito vengono riportati i tre blocchi di codice (funzioni centrali) estratti direttamente dal file principale app.py. Il codice rispetta le direttive formali dello standard di stile PEP 8 (indentazione a 4 spazi, nomi in snake_case, commenti espliciti e organizzazione logica).

5.1 Inizializzazione e Popolamento del Database (init_db)

Questa funzione crea la struttura relazionale dell'applicazione se il database non esiste, inserendo i ruoli iniziali con password protette.

```
def init_db():
    if not os.path.exists(DB_NAME):
        conn = get_db_connection()
        c = conn.cursor()
        # Tabella Utente (con EMAIL come da Doc 2)
        c.execute('''CREATE TABLE Utente (
            id_utente INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT, cognome TEXT, username TEXT UNIQUE,
            password TEXT, ruolo TEXT, email TEXT)''')

        # Tabella Materiale (con POSIZIONE come da Doc 2)
        c.execute('''CREATE TABLE Materiale (
            id_materiale INTEGER PRIMARY KEY AUTOINCREMENT,
            titolo TEXT, autore TEXT, ISBN TEXT,
            categoria TEXT, stato TEXT, posizione TEXT)''')

        # Tabella Prestito (con DATE come da Doc 2)
        c.execute('''CREATE TABLE Prestito (
            id_prestito INTEGER PRIMARY KEY AUTOINCREMENT,
            id_utente INTEGER, id_materiale INTEGER,
            data_inizio TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            data_scadenza TIMESTAMP,
            stato_prestito TEXT,
            FOREIGN KEY(id_utente) REFERENCES Utente(id_utente),
            FOREIGN KEY(id_materiale) REFERENCES Materiale(id_materiale))''')

        # Utenti di test con Password Criptate
        users = [
            ('Admin', 'Muse', 'admin', generate_password_hash('museadmin123'), 'amministratore'),
            ('Marco', 'Bibliotecario', 'biblio', generate_password_hash('musebiblio123'), 'bibliotecario'),
            ('Enrico', 'Senter', 'studente', generate_password_hash('musestudente123'), 'utente')
        ]
        c.executemany('INSERT INTO Utente (nome, cognome, username, password, ruolo) VALUES (?, ?, ?, ?, ?)', users)
```

5.2 Controllo delle Credenziali di Accesso (login)

Riceve le informazioni dal form e utilizza le funzioni di werkzeug.security per convalidare l'accesso in modo sicuro.

```
# --- LOGIN/LOGOUT ---
@app.route('/', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        pwd = request.form['password']
        conn = get_db_connection()
        user = conn.execute('SELECT * FROM Utente WHERE username = ?', (username,)).fetchone()
        conn.close()
        if user and check_password_hash(user['password'], pwd):
            session.update({'id': user['id_utente'], 'user': user['username'], 'ruolo': user['ruolo']})
            return redirect(url_for('dashboard'))
        flash('Username o Password errati!!')
    return render_template('login.html')

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))
```


5.3 Registrazione del Prestito e Scadenza Temporale (prenota)

Questa funzione implementa l'azione di prenotazione: sposta lo stato del materiale, calcola la scadenza esatta a 15 giorni e archivia la transazione.

```
@app.route('/gestione_prestiti')
def gestione_prestiti():
    if session.get('ruolo') not in ['amministratore', 'bibliotecario']: return redirect(url_for('dashboard'))
    conn = get_db_connection()
    query = '''
        SELECT p.*, u.username, u.nome, u.cognome, m.titolo
        FROM Prestito p
        JOIN Utente u ON p.id_utente = u.id_utente
        JOIN Materiale m ON p.id_materiale = m.id_materiale
        ORDER BY p.data_inizio DESC
    '''
    prestiti = conn.execute(query).fetchall()
    conn.close()
    return render_template('gestione_prestiti.html', prestiti=prestiti)

if __name__ == '__main__':
    app.run(debug=True)
```

Calcolo data scadenza sommandovi 15 giorni dall'istante corrente

scadenza = (datetime.now() + timedelta(days=15)).strftime('%Y-%m-%d %H:%M')

1. Aggiorna lo stato del materiale per escluderlo da altre prenotazioni contemporanee

conn.execute('UPDATE Materiale SET stato = "In Prestito" WHERE id_materiale = ?', (id,))

2. Inserisce la transazione associandola all'ID dell'utente in sessione

conn.execute("INSERT INTO Prestito (id_utente, id_materiale, data_scadenza, stato_prestito)
VALUES (?, ?, ?, ?)", (session['id'], id, scadenza, 'Attivo'))

DOCUMENTO 4

DOCUMENTO 4 – COLLAUDO FINALE E MESSA IN ESERCIZIO

1. Piano di Collaudo (Matrice di Collaudo)

In questa fase finale del progetto ho pianificato i test per verificare che tutti e 13 i Requisiti Funzionali (RF) definiti all'inizio della mia relazione (Documento 1) rispondessero senza errori all'interno dell'applicazione. Di seguito è riportata la matrice utilizzata per associare ogni requisito a uno o più casi di test specifici.

ID Requisito	Descrizione Requisito Originale	ID Test	Azione / Input di Test	Risultato Atteso
RF-01	Login: Gli utenti devono poter accedere con username e password. Il sistema deve controllare che i dati siano corretti prima di far entrare l'utente.	TC-01	Inserimento di username e password nel form della pagina di accesso.	Verifica dei dati nel DB backend; se corretti, avvio della sessione e reindirizzamento alla Dashboard.
RF-02	Gestione utenti: Per garantire la massima sicurezza ed evitare registrazioni non autorizzate, le utenze e i relativi ruoli vengono inseriti e gestiti direttamente all'interno del codice di inizializzazione (init_db).	TC-02	Controllo delle utenze e dei ruoli creati automaticamente sul database all'avvio.	Inserimento sicuro e protetto delle credenziali (amministratore/lettore) tramite codice backend.
RF-03	Inserimento materiali: Il sistema deve permettere di inserire nuovi libri o materiali con tutte le informazioni principali come titolo, autore, ISBN, categoria e posizione.	TC-03	Compilazione ed invio del modulo "Aggiungi materiale" con tutti i campi previsti.	Salvataggio del nuovo record nel DB e corretta memorizzazione di ISBN e posizione.
RF-04	Modifica materiali: Le informazioni dei materiali devono poter essere aggiornate quando necessario.	TC-04	Modifica dei campi testuali di un libro esistente tramite l'apposito form di aggiornamento.	Scrittura delle nuove informazioni sul database e aggiornamento della riga interessata.
RF-05	Eliminazione materiali: I materiali non più presenti devono poter essere rimossi dal sistema (solo dall'amministratore).	TC-05	Click sul pulsante di rimozione di un libro (operazione visibile ed eseguibile solo da account amministratore).	Cancellazione immediata e permanente della riga corrispondente dal database.
RF-06	Ricerca: L'utente deve poter cercare i materiali in modo semplice usando titolo, autore, ISBN o categoria.	TC-06	Digitazione di una stringa o di un codice numerico nella barra di ricerca del catalogo.	Filtraggio in tempo reale dei record visibili a schermo in base alla chiave inserita.

RF-07	Visualizzazione catalogo: Il sistema deve mostrare tutti i materiali presenti nella biblioteca con le informazioni principali.	TC-07	Caricamento della pagina principale del catalogo dall'interfaccia utente.	Generazione di una tabella completa che elenca tutti i libri con i rispettivi dati.
RF-08	Prestiti: Il sistema deve registrare quando un libro viene preso in prestito, da chi e per quanto tempo.	TC-08	Click sul pulsante "Prenota" in corrispondenza di un materiale in stato disponibile.	Generazione di un record di prestito associato all'ID utente attivo, con scadenza calcolata a 15 giorni.
RF-09	Restituzioni: Quando un libro viene restituito, il sistema deve aggiornare automaticamente la sua disponibilità.	TC-09	Click sul pulsante "Restituisci" da parte dello staff nel pannello di controllo.	Modifica automatica del record; il materiale torna immediatamente libero per nuove prenotazioni.
RF-10	Stato materiali: Ogni materiale possiede uno stato dinamico aggiornato in tempo reale. I valori gestiti sono 'Disponibile' e 'In Prestito'.	TC-10	Verifica della variazione del flag di stato durante i processi di prenotazione e resa del materiale.	Passaggio automatico e istantaneo dello stato del libro tra i valori 'Disponibile' e 'In Prestito'.
RF-11	Report: Il sistema deve generare report su prestiti, disponibilità e statistiche generali.	TC-11	Accesso alla Dashboard principale del sistema.	Elaborazione di query di conteggio sul database e visualizzazione a schermo dei report numerici aggregati.
RF-12	Storico: Il sistema garantisce la tracciabilità di tutte le transazioni. Quando un materiale viene restituito, il record viene archiviato modificando lo stato in 'Restituito'.	TC-12	Controllo delle righe della tabella dei prestiti nel database in seguito a una restituzione avallata.	Il record non viene rimosso fisicamente, ma aggiornato con lo stato 'Restituito' per mantenere la cronologia.
RF-13	Logout: L'utente deve poter uscire in sicurezza dal sistema.	TC-13	Click sulla voce "Esci" posizionata all'estrema destra della barra di navigazione.	Distruzione completa dei dati memorizzati in sessione e rinvio restrittivo al modulo di login.

2. Report di Collaudo

Tutti i test pianificati sono stati eseguiti all'interno dell'ambiente di sviluppo locale. I risultati ottenuti sono stati per lo più positivi (PASS). Di seguito si documenta il resoconto dettagliato delle risultanze effettive:

- **RF-01 (Login) – RISULTATO: PASS**
 - Test eseguito: Accesso effettuato inserendo lo username admin e la password admin123. Il sistema applica la funzione di cifratura sicuro ed effettua il controllo. I dati coincidono e vengo indirizzato alla Dashboard. Inserendo una password errata l'accesso viene respinto.
- **RF-02 (Gestione utenti) – RISULTATO: PASS**
 - Test eseguito: Analisi del database all'avvio dell'applicazione. Le utenze fisse e i loro livelli di accesso (amministratore e lettore) sono stati creati direttamente tramite il codice della funzione init_db(). Non sono presenti form di registrazione esterni, bloccando accessi non autorizzati.
- **RF-03, RF-04 e RF-05 (Inserimento, Modifica ed Eliminazione) – RISULTATO: PASS**

- Test eseguito: Compilando il modulo di inserimento come amministratore, il sistema aggiunge correttamente il materiale salvando il Titolo, l'Autore, il codice ISBN, la Categoria e la posizione fisica dello scaffale. I form di modifica salvano i dati aggiornati sul file database. Il pulsante di eliminazione cancella la riga all'istante ed è nascosto per gli utenti base.
- **RF-06 e RF-07 (Catalogo e Ricerca) – RISULTATO: PASS**
 - Test eseguito: La tabella /catalogo mostra a schermo tutti i dati. Digitando del testo nella barra di ricerca (ad esempio una categoria come "Scienza" o una stringa dell'ISBN), la pagina applica un filtro escludendo all'istante tutti i libri non corrispondenti.
- **RF-08, RF-09 e RF-10 (Prestiti, Restituzioni e Stato) – RISULTATO: PASS**
 - Test eseguito: Premendo su "Prenota", il sistema associa il materiale all'utente loggato, calcola la scadenza aggiungendo 15 giorni tramite l'istruzione `timedelta(days=15)` e aggiorna in tempo reale lo stato del libro da 'Disponibile' a 'In Prestito'. Premendo su "Restituisci" dal pannello dello staff, lo stato del materiale viene ripristinato automaticamente su 'Disponibile'.
- **RF-11 (Report) – RISULTATO: PASS**
 - Test eseguito: All'ingresso nella Dashboard, l'applicazione esegue correttamente i calcoli tramite funzioni SQL aggregate. Sullo schermo vengono mostrati i blocchi riassuntivi numerici aggiornati (29 Materiali Totali, 1 Prestito Attivo, 3 Utenti Registrati).
- **RF-12 (Storico) – RISULTATO: PASS**
 - Test eseguito: Ispezione della tabella Prestito all'interno del database dopo aver effettuato la restituzione di un libro. La riga corrispondente non viene eliminata dal database, ma il campo `stato_prestito` viene modificato da "Attivo" a "Restituito", preservando lo storico delle transazioni passate.
- **RF-13 (Logout) – RISULTATO: PASS**
 - Test eseguito: Click sul tasto "Esci". La sessione di lavoro viene azzerata tramite `session.clear()` e vengo riportato al login. Provando ad utilizzare le frecce direzionali del browser per tornare indietro, il server intercetta la mancanza di credenziali e impedisce l'accesso.

3. Bug riscontrati ed Errori Iniziali (Bug Fixing)

Durante le fasi di sviluppo individuale di questo applicativo ho commesso alcuni errori iniziali di impostazione, sia a livello logico che di codice, che ho dovuto correggere per rendere stabile il software:

1. Errore iniziale di layout grafico nei Mockup
 - Mio errore: Durante la stesura del Documento 2 (punto 4.3), avevo disegnato per sbaglio nei wireframe grafici un menu di navigazione verticale posizionato sulla sinistra della pagina.
 - Come ho risolto: Sviluppando i file HTML reali mi sono accorto che l'applicazione risultava molto più ordinata e pulita utilizzando una barra di navigazione scura fissa posizionata in alto (Navbar). Ho provveduto a correggere i disegni della relazione per renderli coerenti con il sito vero.
2. Confusione sulla versione di Python nelle note della documentazione
 - Mio errore: Nelle prime bozze del Documento 3 avevo scritto per distrazione di aver utilizzato una versione inesistente di Python (la 3.14.5 datata 2025).
 - Come ho risolto: Ho verificato la configurazione del mio sistema digitando `python3 --version` nel mio terminale e ho corretto il dato inserendo la versione reale che sto usando per far girare Flask (Python 3.11).
3. Bug del database che si cancellava a ogni riavvio
 - Mio errore: Inizialmente la funzione `init_db()` azzerava e ricreava le tabelle ogni volta che avviavo il file `app.py`, causandomi la perdita costante di tutti i nuovi libri aggiunti o dei prestiti inseriti durante i test.

- Come ho risolto: Ho aggiunto un controllo condizionale integrando il modulo nativo os. Ora il codice esegue l'istruzione `if not os.path.exists(DB_NAME):`, facendo nascere il database solo la prima volta e lasciando intatti i dati nei riavvii successivi del server
4. Bug delle chiavi estere disattivate in SQLite
- Mio errore: Durante i test mi ero accorto che il sistema mi permetteva di registrare un prestito associandolo a un codice materiale inventato o non esistente, rompendo l'integrità relazionale del database.
 - Come ho risolto: Studiando la documentazione ho scoperto che SQLite disattiva i vincoli sulle chiavi estere per impostazione predefinita. Ho risolto inserendo il comando esplicito `conn.execute("PRAGMA foreign_keys = ON;")` all'interno della funzione `get_db_connection()`.

4. Istruzioni di Messa in Esercizio

Per installare, configurare ed eseguire l'applicazione in un ambiente di esercizio locale controllato (Localhost), ho seguito questa procedura passo-passo da terminale:

1. Posizionamento nella directory: Aprire il prompt dei comandi o il terminale e spostarsi all'interno della cartella principale del progetto: `cd Desktop / biblioteca_muse`
2. Installazione dei pacchetti: Scaricare ed installare il framework Flask (che si occupa autonomamente di integrare i moduli per la sicurezza delle password) tramite il gestore di pacchetti standard di Python:
`python3 -m pip install flask` (sul terminale macOS)
3. Primo avvio e generazione del DB: Eseguire il file principale in Python per attivare il server web di backend e consentire al codice di forgiare automaticamente il file locale del database `biblioteca_muse.db`:
`python app.py`
4. Utilizzo dell'applicazione: Aprire un qualsiasi browser internet (Chrome, Edge o Safari) e digitare l'indirizzo di rete locale predefinito: `http://127.0.0.1:5000/`

5. Manuale Utente

L'interfaccia dell'applicazione è stata sviluppata per essere immediata. Tutte le funzionalità principali sono raggiungibili tramite la barra di navigazione fissa scura posizionata in alto.

5.1 Accesso al Sistema (Login)

- All'apertura dell'applicazione compare la schermata di accesso centrale.
- L'utente deve inserire il proprio username (es. admin) e la password di sicurezza (es. admin123) e fare clic sul pulsante [Entra nel Sistema]. In caso di errore nelle credenziali, un banner avviserà del fallimento impedendo l'ingresso.

5.2 Pannello Principale (Dashboard e Report)

- Dopo l'autenticazione, il sistema indirizza l'utente sulla Dashboard. Questa pagina mostra un messaggio di benvenuto personalizzato e genera in automatico i report statistici generali richiesti (i contatori dinamici che mostrano il totale dei materiali inseriti, dei prestiti attivi e degli utenti registrati nel database).

5.3 Consultazione del Catalogo e Funzione di Ricerca

- Cliccando sulla voce [Catalogo] presente nella barra superiore, si accede alla tabella completa di tutti i materiali in possesso del MUSE. Per ciascun libro vengono mostrati l'autore, il codice ISBN, la categoria e la collocazione fisica in archivio (Posizione).
- In cima alla tabella è presente una barra di testo: basta digitare una parola chiave (un nome, una categoria o parte del codice ISBN) per filtrare in tempo reale l'elenco dei libri a schermo.

5.4 Eseguire una Prenotazione (Scenario Utente/Lettore)

- All'interno della tabella del catalogo, se un libro possiede lo stato verde "Disponibile", a destra della riga apparirà il pulsante operativo [Prenota].
- Facendo clic su di esso, il sistema registra l'assegnazione. La pagina si aggiorna modificando lo stato del libro in "In Prestito", nascondendo il pulsante per impedire ad altre persone di selezionarlo contemporaneamente.

5.5 Gestione dei Prestiti e Restituzione (Scenario Staff/Admin)

- Gli utenti dotati di privilegi amministrativi visualizzano nella barra superiore il pulsante aggiuntivo [Gestione Prestiti].
- Questa sezione mostra il registro storico delle movimentazioni, includendo i dettagli su chi ha preso il libro e la data di scadenza dei 15 giorni (scelto a caso).
- Quando il lettore riconsegna fisicamente il volume, l'operatore fa clic sul pulsante azzurro [Restituisci]: il sistema chiude la transazione archiviandola come 'Restituito' e ripristina all'istante lo stato del libro in "Disponibile" all'interno del catalogo.
- Per uscire dal sistema in modo sicuro, basta fare click sul pulsante [Esci] posto all'estrema destra della barra dei menu.